

FREUID Challenge 2026 — Technical Report

1. Introduction

We address identity-document fraud detection as a binary image-classification problem: each document image is assigned a continuous fraud score (higher = more likely fraudulent), evaluated with the challenge’s tail-focused DET metric (APCER at low BPCER). The task exhibits a strong domain shift between the training data and the (public/private) test data — including print-and-capture recaptures and document types under-represented in training — which drives our design choices toward strong local-artifact modelling at inference time.

2. Method

Backbone. `maxvit_base_tf_512.in21k_ft_in1k` (timm), a hybrid conv + window/grid-attention model at 512×512 . In an earlier fold-0 sweep of 30+ timm backbones, MaxViT @ 512 clearly dominated (mid-size hybrid + high resolution beat larger pure-conv / pure-ViT models that overfit the digital domain), which is why it is our final backbone.

Training data. All competition training images with the 5 stratified folds merged (no held-out validation — in-domain OOF is saturated and does not discriminate models), plus **self-generated pseudo-labels**: confident predictions on the public test images (score $< 0.05 \rightarrow$ bona-fide, $> 0.95 \rightarrow$ fraud) added to the training set (semi-supervised). **No external datasets are used.**

Objective / optimisation. BCEWithLogitsLoss on a single logit; AdamW (lr $1e-4$, weight-decay $1e-5$); cosine schedule with 1-epoch warmup; bf16 autocast; channels-last. Image size 512, batch size 10, 12 epochs.

Augmentation. “V1”: horizontal-flip, vertical-flip, rotate($\pm 20^\circ$), then resize the whole document to 512. (Crop-based *training* was tried and hurt; crops help only at *test* time — see §5.)

Checkpoint ensemble. We save epochs 10, 11, and 12 and ensemble all three; averaging the three neighbouring epochs denoises the pseudo-label-trained model.

3. Inference

For every test image each of the 3 checkpoints is scored with **4-view edge-crop test-time augmentation**:

- from a $1.15\times$ up-scaled image we take the **4 edge-midpoint crops** of a 3×3 grid (top / left / right / bottom centre) at 512×512 — no flip, no native view, no corner crops;
- the 4 crop probabilities are averaged, then the 3 checkpoints are averaged $\rightarrow$ one fraud score per image = $4 \times 3 = 12$ **forward passes/image**.

Rationale: fraud cues (font/MRZ/splice/GenAI edits) are *local*; a full-image resize blurs them, whereas averaging local crops preserves them. A leaderboard sweep showed the **edge crops carry the whole signal** — corner crops add noise, and flips / the native view / a full 3×3 grid are redundant on top of the 3-checkpoint ensemble. This 12-pass scheme is both **more accurate** than the 40-view dense-crop grid (0.00115 vs 0.00145, §4) **and** $\sim 10\times$ cheaper, which is what brings the hidden-test-set runtime within the reproducibility budget (§5). Inference is deterministic and needs no network (pretrained=False; weights from local checkpoints).

4. Results

Public leaderboard (AuDET, lower = better):

Configuration	Views/ img	Public
Single checkpoint (e12), dense-crop TTA (40 views)	40	0.00178
3-checkpoint ensemble, dense-crop TTA (40-view D2 grid)	120	0.00145
3-checkpoint ensemble, 5-crop $\times$ flip TTA	12	0.00174
2-checkpoint (e11+e12), centre+4 edge-crops	10	0.00129
3-checkpoint ensemble (e10+e11+e12), 4 edge-crops — final	12	0.00115

The final configuration is both the **best-scoring** and, at 12 forward passes/image, well within the runtime budget (§5). Adding more views (full 3×3 grid, flips, native view) does not improve on 0.00115 — the edge crops saturate the signal for this ensemble.

5. Reproducibility

- **Code / weights frozen** at the tagged commit (see the pinned Kaggle reply for the exact SHA); no model, architecture, or training-code changes after the freeze date.

- **Deterministic, network-free inference** via the provided Docker image, run with `--network none`; weights are baked into the image (Git LFS in the repo).
- **Data sources:** competition training set + self-generated pseudo-labels on the public test set. No external data, no runtime downloads.
- **Training entrypoints:** `train/` (`make_folds.py`, `pseudo_gen.py`, `train_full.py`) reproduce the checkpoints from the competition data; commands are documented in `train/README_TRAIN.md`.
- **Hardware:** single NVIDIA H100 (80 GB), CUDA 12.6, PyTorch 2.7.1, timm 1.0.27.
- **Measured runtime (12 passes/image, `--network none`, `bf16`, `channels-last`).** The **plain-default** run (`--bs 16 --workers 0` — in-process decode, so no `--shm-size` flag is needed and a bare docker run works) scores the full **7,821-image public set in 428 s (7 min)** on one H100 → **≈2.0 h for the 134,997-image hidden test set on one H100**. A multi-worker path (`--shm-size=8g --workers 8`) is ~15 % faster (372 s → ≈1.8 h).
- **A100 budget (≤6 h requirement).** Extrapolating the default by the H100→A100 slowdown for this `bf16` workload (memory-bandwidth ratio ≈1.7× ... compute-peak ratio ≈3.2×; a realistic ≈2.0×):

A100/H100 factor	hidden-set runtime (default)
2.0× (realistic)	≈ 4.1 h
2.5× (pessimistic)	≈5.1 h
3.0× (compute-peak worst case)	≈6.1 h

i.e. **within the 6-hour single-A100 limit at realistic slowdowns** (≈4 h, ~2 h margin); the faster `--workers 8` path or sharding /data across GPUs gives extra headroom against the compute-peak worst case.

6. Notes / limitations

The gains at the extreme low-FPR tail are small in absolute terms (a handful of tail images on the 5% public split), so ranking there is sensitive. The final 4-edge-crop × 3-checkpoint scheme was selected as the **best-scoring configuration that also meets the 6-hour single-A100 reproducibility runtime limit** (12 forward passes/image); adding more views, flips, the native resize or the corner crops does not improve on it. The pseudo-labels are a domain-adaptation bet for the private set: they do not help the public score (the non-pseudo base scores better on public), so their benefit is only realised on the shifted private test.